

Simulation Engine, Dashboards & Automation

Three Monte Carlo scenarios (each with a real methodology bug caught and fixed), five Power BI dashboards, and a scheduled, idempotent pipeline.

Prepared by	Neelam Lahre		
Tools added	Power BI Desktop, Windows Task Scheduler		
Scripts	python/04_simulate.py, 04b_simulate_s02.py, 04c_simulate_s05.py, run_pipeline.py		
Guide reference	Solutioning Guide v1.0, Stages 10, 11 and 12		
Status	Stages 1-12 complete -- the full 12-stage build.		

3 / 8	5	3	Hourly
scenarios built in full	dashboards + Scenario Studio	schema gaps found & fixed	scheduled, idempotent pipeline

Stage 10 — Monte Carlo Simulation Engine

Goal: ten thousand futures instead of one guess. Every scenario below hit a real methodology problem on its first run -- each one caught by a proof-check built for exactly that purpose, not glossed over.

S-01 — Product mix shift +10pp to Savings & Wealth

Bug found	Fix	Final result
'UNMAPPED' (173 seeded junk category rows) counted as a 5th real product segment in the baseline query	Added AND p.product_category <> 'UNMAPPED' to the baseline query and to vw_nbp_margin itself	Baseline 8.952% -> p05/p50/p95 4.83% / 7.35% / 9.86%. Central impact -160 bps. NBP at risk S\$13,071,332

S-02 — IP rider co-pay introduction

Bug found	Fix	Final result
Only 8 distinct copay_pct values exist, heavily skewed to 0 -- a linear fit gave a result backwards from the business hypothesis	Discarded the linear fit; bootstrapped the REAL 10%-copay cohort directly (271 real policies) instead of extrapolating a slope	Baseline lapse 9.86% -> projected p05/p50/p95 2.95% / 5.17% / 7.38%. Lapse change -470 bps. Premium impact +S\$526,666

S-05 — Assessor threshold-gaming detection

Bug found	Fix	Final result
Rogue assessor's near-limit rate was diluted against their FULL lifetime claim history -- 60 injected claims couldn't move a career-long average	Switched to a 40-claim rolling window (what a real monitor would actually watch), not a lifetime cumulative rate	Detection rate 0.9% -> 100%. Median claims to detect: 57 -> 9. Leak before detect: S\$70,841. Leak prevented: S\$353,983

Housekeeping: the discarded/buggy runs (run_id 1, 3, 5) were deleted from scenario_run/scenario_result -- superseded methodology, not legitimate history.

Documented backlog — S-03, S-04, S-06, S-07, S-08

Ref	Question	Technique named in the guide
S-03	Care delay +6 months, portfolio-wide	Severity-escalation model on claim-stage transitions
S-04	Claims dispute goes viral	Sentiment-to-lapse / sentiment-to-acquisition regression
S-06	Top 5% agents resign this quarter	Agent-level attrition scoring + premium-at-risk roll-up
S-07	Competitor cuts protection pricing 15%	Competitive response model, channel switching

Proved: all three built scenarios saved to `scenario_run/scenario_result` with seed and iteration count recorded -- reproducible on demand, per the guide's own model-governance requirement.

Stage 11 — Power BI Dashboards

Goal: put it in front of the business. Composite model -- DirectQuery on facts and KPI views, Import on dimensions and the scenario store.

Three more schema gaps found while building the dashboards

Dashboard need	Missing column(s)	Fix
S-02's price-elasticity model	rider_attached_flag, ip_plan_tier, rider_premium_before/after_sgd	ALTER TABLE + backfill script (03c_backfill_rider_fields.py)
D-5 sentiment vs. lapse scatter	customer_sentiment_score, nps_score	ALTER TABLE + backfill script (03d_backfill_sentiment.py)
D-6 productivity quadrant	agent_productivity_score, agent_tenure_months	ALTER TABLE + backfill script (03e_backfill_agent_fields.py)

None of these five columns were in Stage 8's original `FACT_COLS` list -- matching the guide's own limited list. All three ALTER TABLE statements are now also folded into `sql/01_schema.sql` for a fresh build, with `sql/03_alter_columns.sql` keeping the historical record of what actually ran.

Scenario Studio — the simulation made interactive

A live 'Mix Shift %' slider (0-25), with the sensitivity coefficient calibrated from the real S-01 result (-160 bps for a 10pp shift = -0.16 fraction-per-unit-shift), not an assumed number. Moving the slider to 10 reproduces the script's own p50 (~7.35% margin, ~\$13M impact) instantly, with no re-run of the actual Python simulation -- exactly the guide's own design intent.

Row-Level Security

Agency Leader and Executive roles built and tested (temporarily substituting a real `agency_unit` value for `USERPRINCIPALNAME()` to prove the filter mechanism, then reverted to the production formula) -- confirmed an Agency Leader viewing as AU-20 sees only AU-20 data. Channel Head is scoped out: the guide's own version needs a `sec_user` mapping table that was never part of this dataset.

Five dashboards built

Dashboard	Key visuals	Notable finding
D-4 Executive Cockpit	Margin gauge, low-margin mix trend, APE trend, channel matrix	Gauge confirms ~9% blended margin against a 15% target line
D-5 Claims & Customer Trust	TAT histogram, decline-reason bars, dispute trend, sentiment scatter	Lapsed customers cluster at visibly lower sentiment scores
D-6 Agency Performance	Productivity quadrant, attrition heat map, premium-at-risk card	110 agents (~9%) at <code>attrition_risk</code> >= 0.5, \$37.8M premium at risk
D-7 Risk & Fraud Control	Z-score ranking, above-limit rate, SoD breach log, alert queue	Alert queue (<code>z</code> >= 2) matches the same assessors Stage 9/10 flagged
D-8 Scenario Studio	Mix Shift slider, live margin/impact cards	Calibrated directly from the real S-01 Monte Carlo output

One documented gap: the guide's D-4 'APE vs plan waterfall' was substituted with a plain APE trend -- no budget/plan/target dataset exists anywhere in this build to compare against.

Proved: every dashboard's headline number cross-checks against a script output from an earlier stage (Total APE = \$818M matches the S-01 baseline; the alert queue matches Stage 10's fraud simulation) -- the dashboards are reading the same certified truth, not a second, drifting copy of it.

Stage 12 — Automate, Monitor, Hand Over

Goal: a pipeline nobody trusts is a pipeline nobody uses.

Real gap found: Stage 8's load script only ever appended -- scheduling it as-is would duplicate the warehouse on every run. Fixed by adding a clear-before-reload step to python/03_load.py, making the whole pipeline safely re-runnable.

python/run_pipeline.py chains profiling, cleaning, load, claims load and all three backfill scripts with fail-fast logging to logs/pipeline.log. Deliberately excludes the simulation scripts (04_simulate.py etc.) from the schedule -- each run inserts a new scenario_run row, so an hourly re-run would spam the scenario store rather than keep data fresh. Simulations stay a deliberate, on-demand action.

Check	Result
Pipeline run (python/run_pipeline.py)	"Pipeline completed successfully", no FAILED entries in pipeline.log
fact_policy_new_business row count after reload	46,188 -- identical to before, confirming a clean round-trip
fact_claim row count after reload	27,583 -- identical to before
Scheduled task (schtasks /Query)	"Prudential Analytics Pipeline" -- Status: Ready, hourly

Proved: the scheduled task is registered and Ready, and a full clear-and-reload cycle reproduces the exact same row counts -- the pipeline is genuinely idempotent, not just fast the first time.

Day 4 (Stages 10-12) complete -- the full 12-stage build is done. All project files (SQL, Python, Power BI .pbix) are saved and consistent with the live database.